

Architecture of Web Applications

Michael Mrissa

michael.mrissa@upr.si

Univerza na Primorskem

Core of Web Applications

•What does a WebApp do ?

- Provides a service, a functionality

•Yes, but what does it do ?

- Processes data (business logic)
- Accesses persistently stored data (data access)
- Formats data (presentation)
- And provides « side » features (authentication, etc...)

•How it is organized ?

- One (large) piece of code ?
 - monolithic application
- Why monolithic applications are bad practice ?

Core of Web Applications

•How to structure the code ?

- According to the client, project, teams, equipment...

•3 concerns

- Data access / Business logic / Presentation

•Additional concerns

- Authentication, security, privacy, performance, scalability, etc.

•How to create interfaces ?

- See lecture about REST

Core of Web Applications

.Presentation

- User interface (not API) to interact with the app
- Classical GUI (standalone client)
- Web-based (lighter, through browser)

.Data access

- Reads and writes data on physical support
 - Files (XML, binary, CSV...)
 - Database
 - Simple relational DB, NoSQL...
 - Data duplication and distribution

Core of Web Applications

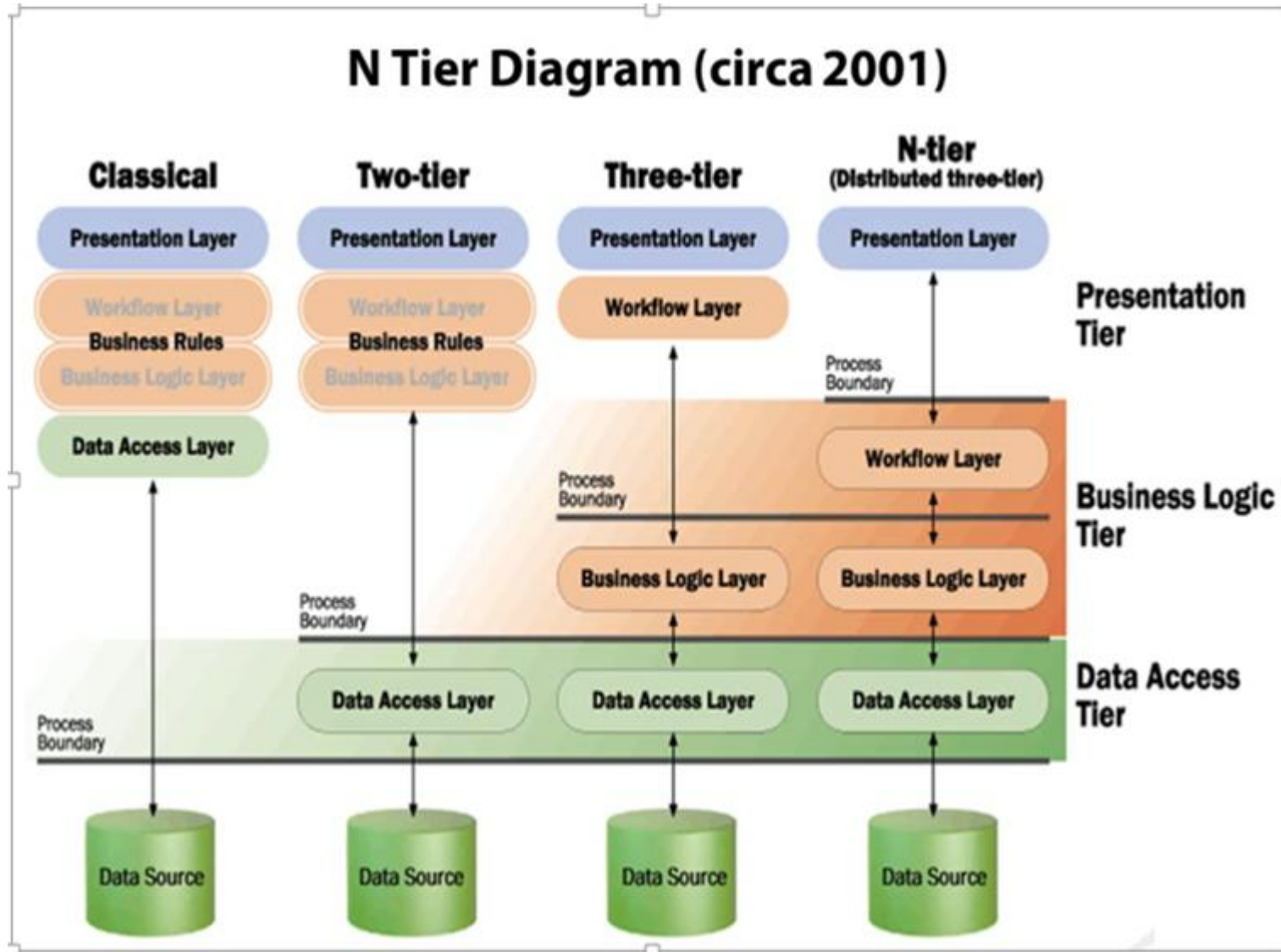
•Business logic

- Provides the added value
 - Ex : How to book a plane ticket
- Offers users with « services »
 - Ex : create a booking, modify, cancel...
 - Different types of interfaces (APIs, REST...)
- Services can be combined → workflows (another problem)

•Need to structure the application

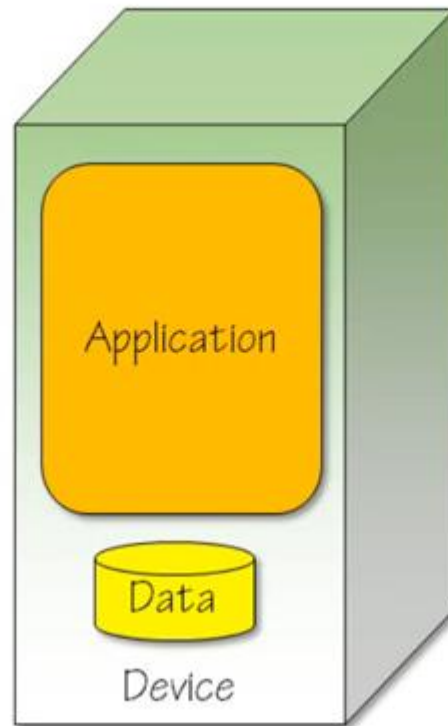
- Tier (physical hardware infrastructure)
- Layer (logical software architecture)
- Sometimes used interchangeably

Core of Web Applications

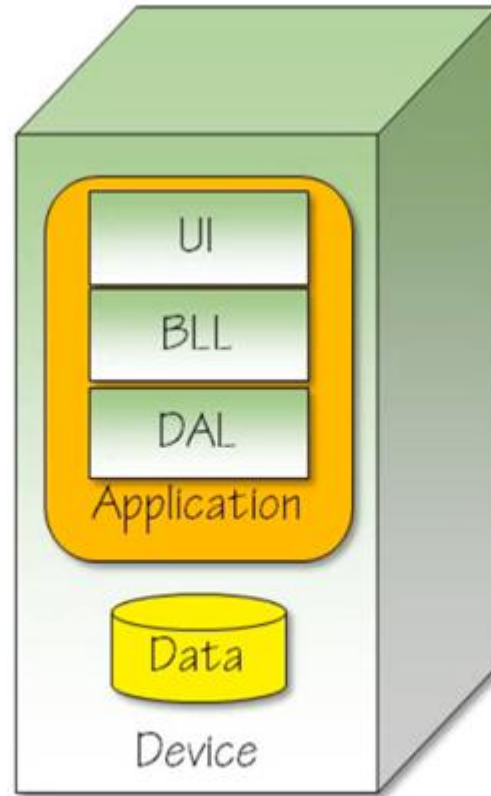


Evolution of Web Applications

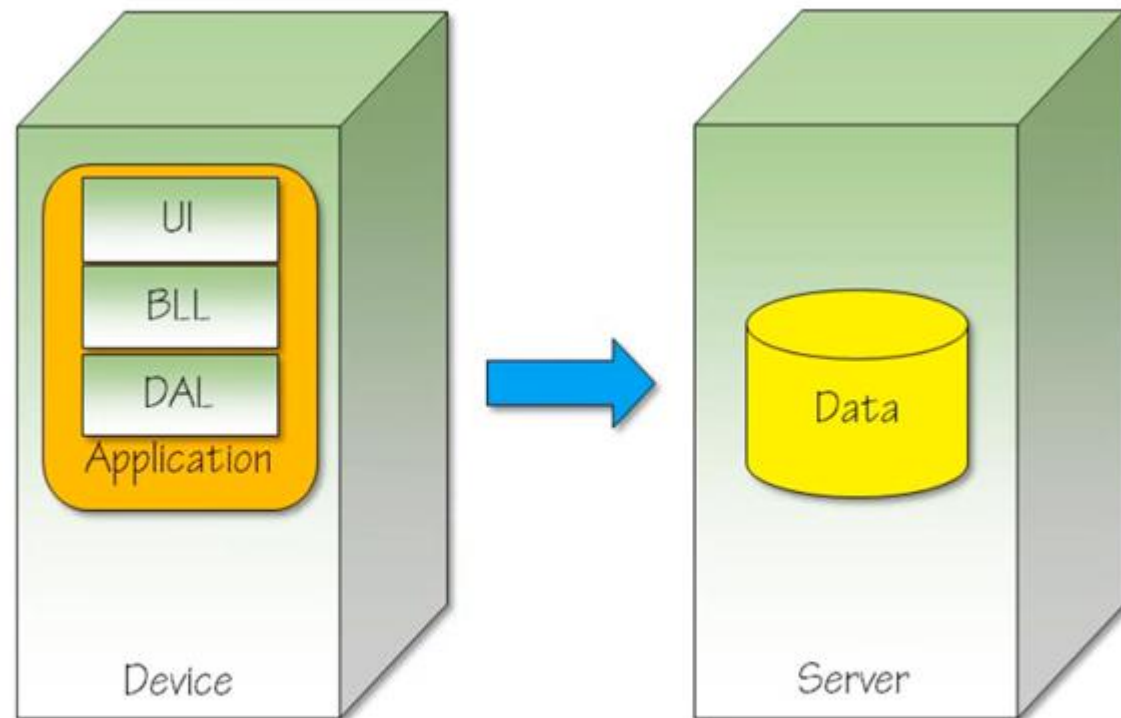
Evolution of N-Tier Applications



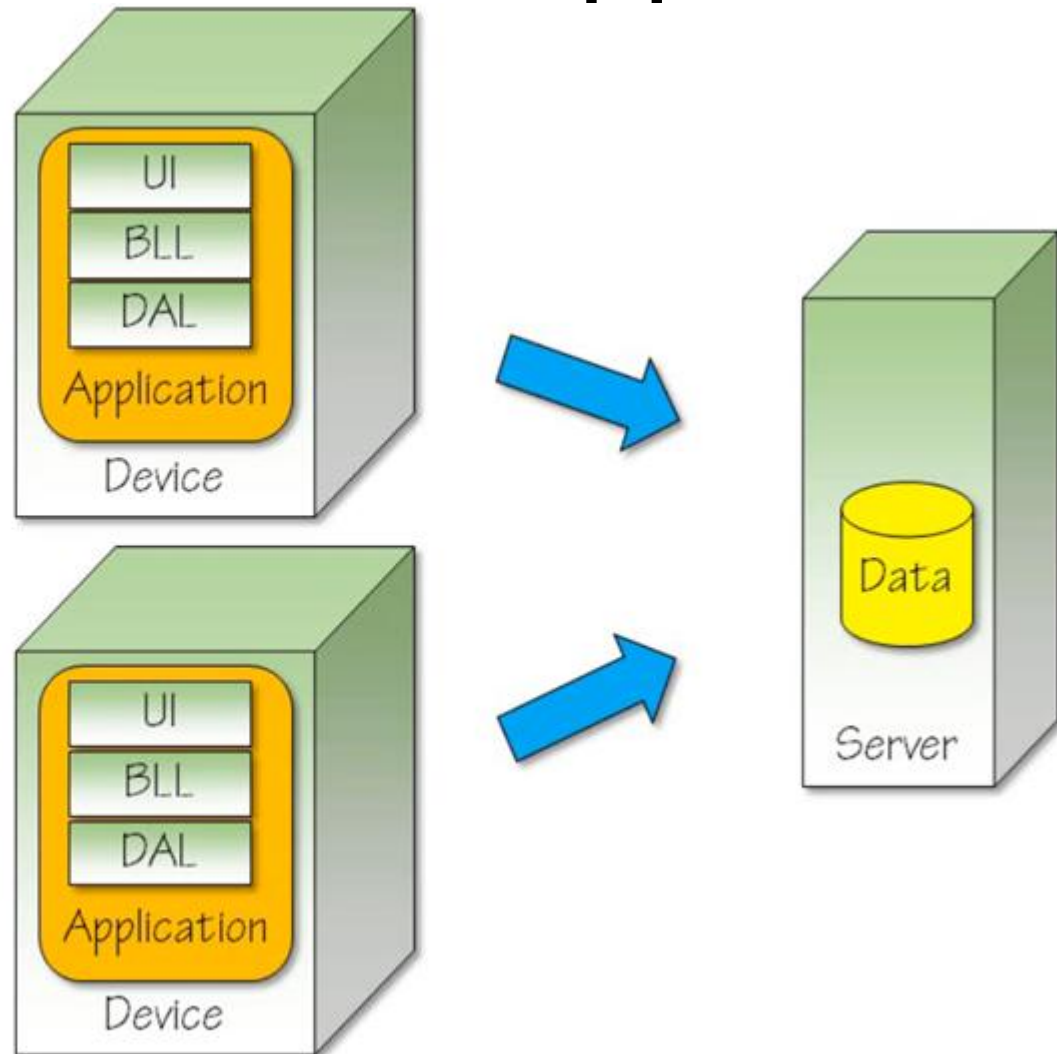
Evolution of Web Applications



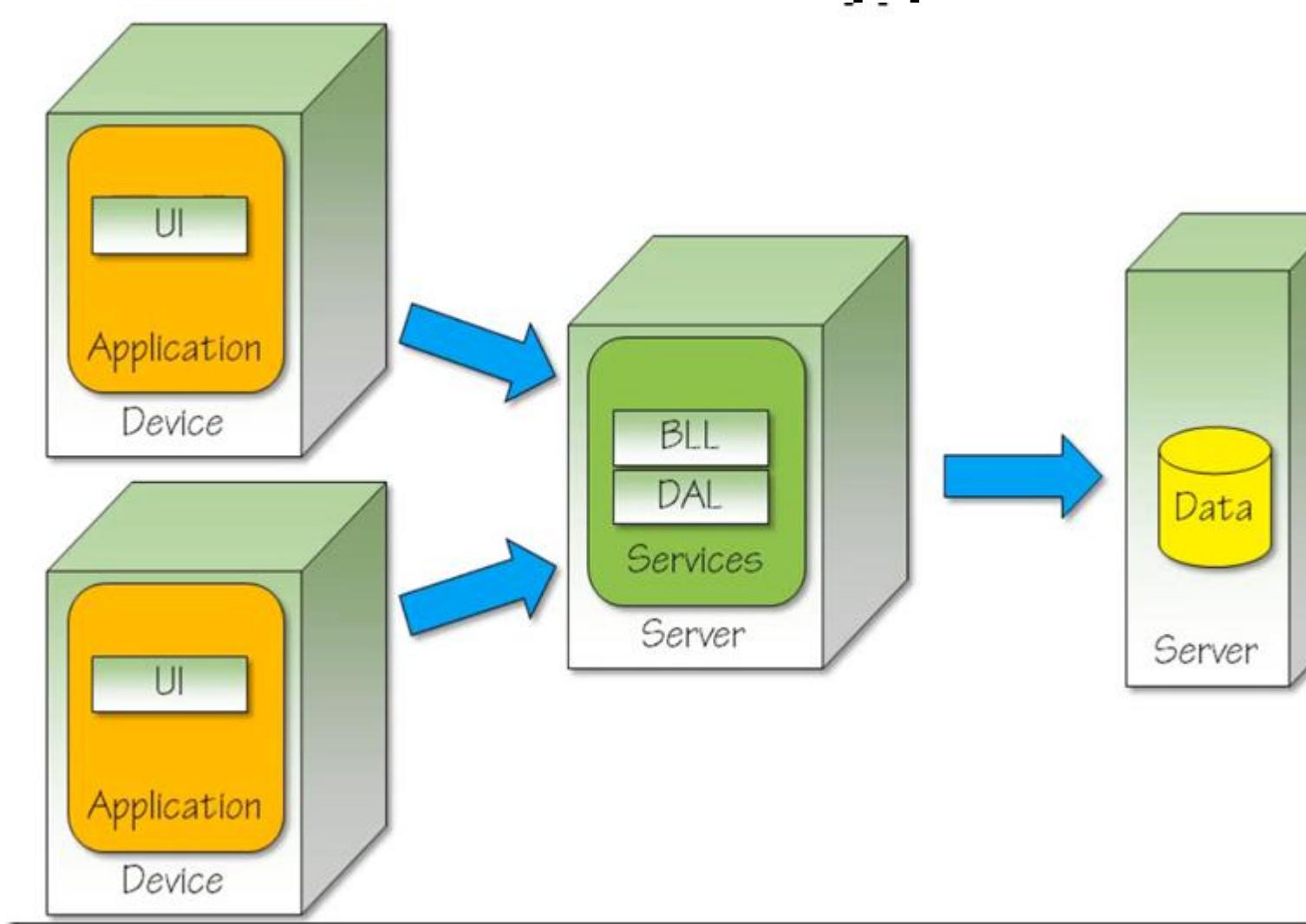
Evolution of Web Applications



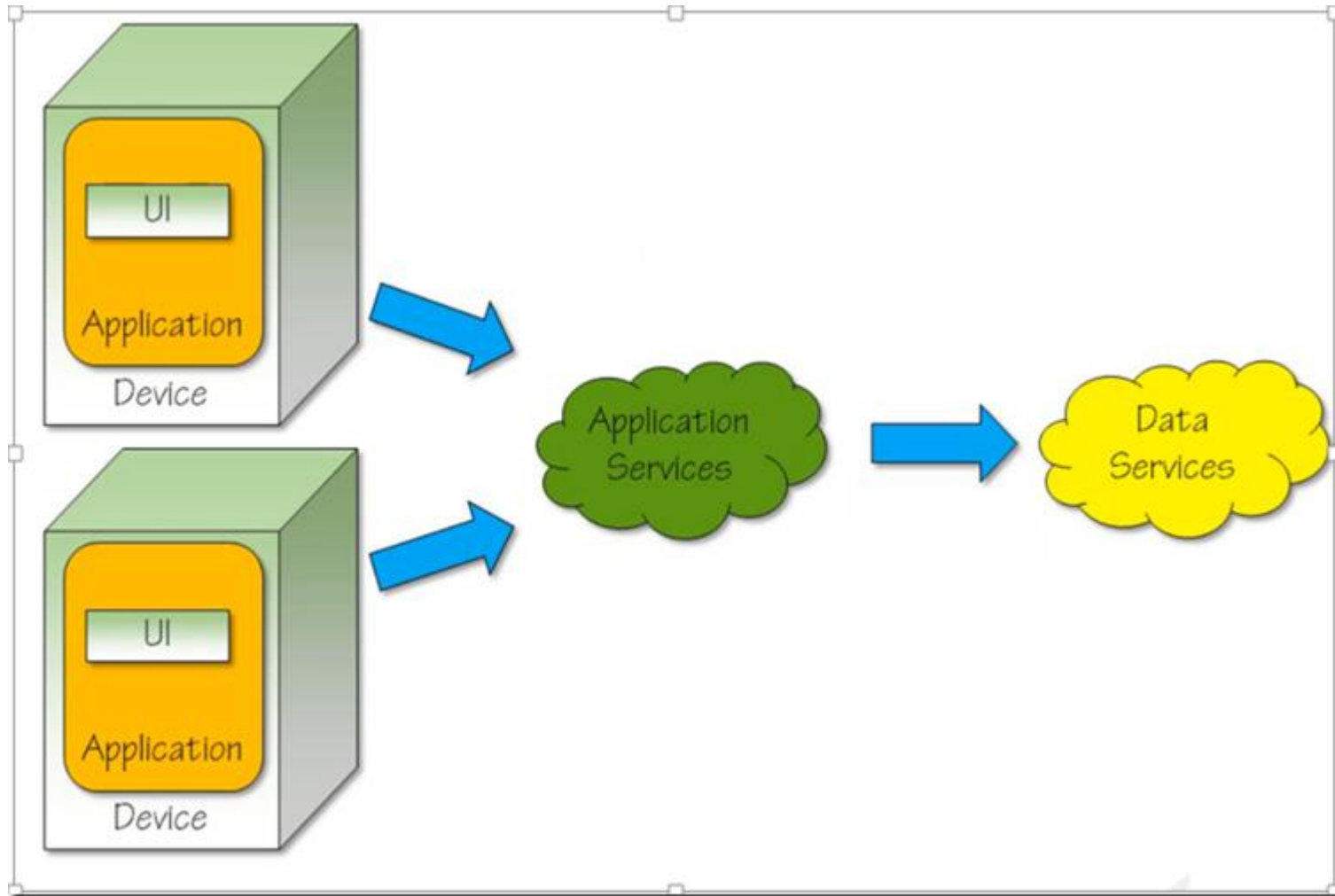
Evolution of Web Applications



Evolution of Web Applications



Evolution of Web Applications



Evolution of Web Applications

•Advantages

- Reuse, productivity, loose coupling
- Externalization easier (cloud comp.)
- Replacement, evolution of component easier
- Load balancing over physical infrastructure

•Drawbacks

- Performance, performance, performance
- Complexity (design and deployment)

Server-side Programming

.Principle

- Program executes on the server
- Sending a dynamically-built resource representation to the client

.Different technologies

- SSI : server Side Includes
 - Inclusion of contents in the page (pre-processing)
- Interpreters integrated into the HTTP server
 - examples : PHP, ASP, Java, Python, NodeJS...
 - Under Apache : additional modules
- CGI : Common Gateway Interface
 - Standardized call to an external program
 - scripts CGI-like : mod_perl / Apache

Exercise

•Design an architecture for your project

- Try different alternatives
- Compare advantages and drawbacks
- Which processing are client-side, which server-side ?
- How to deal with security ?
- How are clients structured ? Browser or other ?
- Optional : what language would you choose ?